

Object-Oriented Programming (OOP)

- OOP stands for object-oriented program.
- Python supports object-oriented program.

Oop allows us to organize programs using:

- classes
- Objects
- Methods
- Properties

* Advantages of OOP.

- Provide a clear structure
- Makes code easier to maintain.
- Makes code reusable.
- Makes debugging easier.
- Helps avoid repetition.
- Allows us to build reusable applications.

* DRY Principle

DRY = Don't Repeat Yourself

∴ It means we should avoid writing the same code repeatedly.

Classes and Objects

- A class is a blueprint for creating objects.
- An object is an instance created from a class.

Example:

class → Fruit

Objects → Apple, Banana, Mango

Creating a class

We use the class keyword.

```
class MyClass:
```

```
    x = 5
```

∴ Here, My class is a class and x is a property.

I. Creating an Object

→ An object is created from a class.

```
p1 = MyClass()
```

```
print(p1.x)
```

Here,

- MyClass = Class
- p1 = Object

II. Multiple Objects

→ We can create many objects from the same class.

```
p1 = MyClass()
```

```
p2 = MyClass()
```

```
p3 = MyClass()
```

```
print(p1.x)
```

```
" (p2.x)
```

```
" (p3.x)
```

__init__() Method

→ __init__() is a special method that runs automatically when an object is created.

∴ It is mainly used to initialize object properties.

```
class Person:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Why use __init__()?

→ Without __init__(), we have to assign properties manually.

→ With __init__(), we can give initial values while creating the object.

```
p1 = Person("Linus", 28)
```

∴ This makes object creation easier.

== Default Values in init ()

→ We can give a default value to a parameter.

```
class Person :
```

```
    def __init__(self, name, age = 18):  
        self.name = name  
        self.age = age
```

∴ If age is not provided, Python uses 18.

== Multiple Parameters

→ init () can have multiple parameters.

```
class Person :
```

```
    def __init__(self, name, age, city, country):  
        self.name = name  
        " • age = age  
        " • city = city  
        " • country = country
```

Self Parameter

Self refers to the current object/instance.

It is used to access the properties and methods belonging to that object.

```
class Person:
```

```
    def __init__(self, name):
        self.name = name
```

```
    def greet(self):
```

```
        print("Hello, my name is" + self.name)
```

Why use self?

Suppose we have two objects

```
p1 = Person("Tobias")
```

```
p2 = Person("Linus")
```

∴ Self tells Python which object's data should be used.

So,

```
p1.printname()
```

∴ uses the data of p1, while:

```
p2.printname() → uses the data of p2.
```

Accessing Properties

Object properties can be accessed using dot notation.

```
car1.brand  
car2.model
```

Example:

```
class Car:
```

```
    def __init__(self, brand, model):  
        self.brand = brand  
        self.model = model
```

```
car1 = Car("Toyota", "Corolla")
```

```
print(car1.brand)  
print(car1.model)
```

* Class Methods

A method is a function that belongs to class.

Methods define the behaviour of objects.

```
class Person:
```

```
    def __init__(self, name):  
        self.name = name
```

```
    def greet(self):
```

```
        print("Hello, my name is" + self.name)
```

∴ call the method:

```
p1.greet()
```

Methods with Parameters

Methods can also accept parameters.

```
class Calculator:
```

```
    def add(self, a, b):  
        return a+b
```

```
    def multiply(self, a, b):  
        return a*b
```